SparkFit System Architecture Document

1. System Overview

SparkFit is a personalized AI Fitness Coach application that leverages React Native (Expo) for the cross-platform frontend and Supabase (PostgreSQL, Auth, Edge Functions) for the backend. The core value proposition is an AI-driven experience powered by the Google Gemini API, which dynamically generates workout plans, provides real-time coaching tips during execution, and context-aware chat capabilities.

2. Frontend Architecture (React Native & Expo)

The frontend is built using Expo and configured to deploy across iOS, Android, and Web platforms.

Core Stack

- **Framework:** React Native with Expo Router/React Navigation.
- **Styling:** React Native localized stylesheets.
- **State Management:** React Context and Local State (Hooks).
- **Backend SDK:** `@supabase/supabase-js` for querying the database and invoking edge functions.

Key Screens & Navigation Flow

The navigation is defined in `AppNavigator.tsx` and partitioned into distinct stacks:

1. **AuthStack:**
   - `WelcomeScreen` : The entry point for non-authenticated users. Handles Email/Password Sign Up and Sign In directly interacting with Supabase Auth.
2. **OnboardingStack:**
   - `AIOnboardingChatScreen` : A conversational AI interface where the user chats with the AI Coach to determine their fitness goals, experience level, and available equipment. This culminates in the generation of a personalized 7-day workout plan via an Edge Function.
3. **MainTabNavigator (Bottom Tabs):**
   - `DashboardScreen` : The daily home base. Displays "Today's Workout" with exercise lists, an estimated duration badge, and a summary insight from the coach.
   - `WorkoutPlanScreen` : Displays a continuous calendar (`react-native-calendars` with Web fallbacks) charting the generated 7-day plan, allowing users to tap future dates to preview sets and reps.
   - `ProgressScreen` : A visual heatmap indicating consistency. Solid green indicates a fully completed workout, while orange indicates partial completion based on the `workout_logs` table.
   - `AICoachScreen` : A standard chat interface directly linked to the AI Coach for general fitness advice.
4. **Workout Flow Stack (Full Screen Modal):**
   - `WorkoutActiveScreen` : The immersive execution interface. Replaces keyboards with quick-tap Stepper Inputs (+/-) for reps and weight. Incorporates a top progress bar, a total timer, an active set timer, automated rest timers (90s default), and dynamic AI coach tips injected per exercise.
   - `MidWorkoutChatScreen` : A context-aware chat modal overlay accessible during the active workout. Automatically passes the current exercise and set context to the AI for immediate, relevant advice.

- `WorkoutSummaryScreen` : Appeared post-workout. Captures final stats, perceived exertion (Subjective Feedback: Easy/Good/Hard), and allows the user to add qualitative workout notes before saving the log to Supabase.

3. Backend Architecture (Supabase)

Supabase provides the fully managed PostgreSQL database, authentication, and serverless Edge Functions running on the Deno runtime.

Database Schema (PostgreSQL)

The database structure is designed for simplicity in the MVP phase, utilizing JSONB columns for flexible data schemas generated by the AI.

- `profiles` :
  - Tied 1:1 to `auth.users` via a Postgres Trigger.
  - Stores user metadata: `goal` , `fitness_level` , `workout_days_per_week` , `equipment` , and `ai_context_summary` (an AI-generated persona summary used in future prompts).
- `workout_plans` :
  - `user_id` (FK to profiles).
  - `plan_data` (JSONB): The exact structured JSON output from Gemini containing the 7-day routine, rest days, exercise names, target sets/reps, and specific `coach_tip` strings for each movement.
- `workout_logs` :
  - The daily tracking table.
  - `logged_data` (JSONB): Stores the actual execution data. For example: `{ "Barbell Squat": {"sets": 3, "reps": [10, 10, 8], "weight": [135, 135, 135], "durations": [45, 42, 48]} }` .
- `workout_feedback` :
  - `perceived_exertion` (Int 1-10) and `text_feedback` capturing the user's subjective difficulty rating at the end of a session.

Row Level Security (RLS) is enabled across all tables, ensuring users can only read, insert, or update their respective data based on `auth.uid() = id` .

Supabase Edge Functions (Deno & Gemini API)

All complex AI logic is offloaded to serverless Edge Functions to protect the Gemini API keys and ensure secure database querying.

1. `generate-workout-plan`
   - **Purpose:** Takes the conversational onboarding transcript, queries the user's profile, and calls the Gemini `gemini-2.5-flash` model.
   - **Output:** Outputs a strict JSON structure containing the 7-day plan. Upserts this JSON directly into the `workout_plans` table and updates the `profiles.ai_context_summary` simultaneously.
2. `chat-coach`
   - **Purpose:** A general-purpose fitness chat endpoint.
   - **Mechanics:** Securely fetches the user's `profiles.ai_context_summary` and their active `workout_plans.plan_data` . It injects this as a `system` instruction to the Gemini model,

ensuring all advice is highly contextual to the user's exact current routine. Supports direct streaming-compatible fetch requests.

3. `chat-onboarding`
    - **Purpose:** A tightly scoped conversational agent specifically designed to extract the necessary parameters (Goal, Level, Days, Equipment) from the user through natural dialogue before triggering plan generation.

4. Current Project State

The SparkFit MVP has been fully implemented and QA tested across major flows.

Completed Milestones:

- User Authentication (Sign up/Login).
- Generative AI Onboarding Flow.
- Supabase Database architecture and RLS policies.
- Full Workout Execution Flow with thumb-friendly Stepper inputs and Automated Rest Timers.
- Macro and Micro Live Tracking Visuals (Progress Bars, Set Timers, Global Timers).
- Comprehensive Workout Logging and State restoration logic on the Dashboard.
- Deep AI Integration: Real-time dynamic `coach_tip` banners, Context-Aware Mid-Workout Chat overlays, and a standalone AI Coach chat interface.
- Calendar Progress Tracking interfaces.
- Web Target Compatibility (Bypassing `react-native-calendars` crashes and fixing React Navigation Linking issues).

Remaining / Next Steps (Post-MVP):

1. **Dynamic Plan Adjustments:** Implement a CRON job or trigger that uses the data in `workout_feedback` and the actual `workout_logs` to prompt Gemini to modify the *next* week's `workout_plans` (e.g., increasing weight targets if marked "Too Easy").
2. **Exercise Library & Videos:** Connect the exercise names generated by Gemini to a static library of form-check videos or GIFs to assist beginners visually.
3. **Advanced Analytics:** Build deeper charting integrations on the Progress screen to graph total volume loads (Weight x Reps) over time for specific exercises like Squat or Bench Press.
4. **Push Notifications:** Implement local or remote push notifications to remind users of their scheduled workouts or notify them when a rest timer completes while the app is backgrounded.
5. **App Store Deployment:** Prepare the standalone application binaries via Expo EAS (Expo Application Services) for deployment to the Apple App Store and Google Play Store.